Capítulo 3. Escribir código

Este trabajo se ha traducido utilizando IA. Agradecemos tus opiniones y comentarios: translation-feedback@oreilly.com

> *Cualquier tonto puede escribir código que una computadora pueda entender. Los buenos programadores escriben código que los humanos pueden entender.*
>
> —Martin Fowler, desarrollador de software británico, autor y conferenciante internacional sobre desarrollo de software

Escribir código es, sin duda, una parte muy importante de la ingeniería de software. Y aunque la programación se enseña ampliamente, los matices de escribir *buen* código no se distribuyen de manera tan uniforme. El hecho de que *puedas* escribir código para resolver un problema no significa que *debas* escribir código para resolver un problema.[1] Con la llegada de la inteligencia artificial y las herramientas de codificación agenticas, el trabajo de un ingeniero de software está evolucionando hacia uno con menos trabajo de desarrollo práctico y más corrección de errores, correcciones y revisión de código generado por

una herramienta de IA entusiasta. Sin embargo, para ser bueno revisando código, debes ser bueno escribiendo código. Puede parecer contradictorio, pero escribir código bueno y limpio es una habilidad invaluable en el mundo de la IA.

Para bien o para mal, los desarrolladores suelen tener opiniones muy firmes sobre lo que constituye un buen código o un mal código, pero las métricas y las herramientas pueden darte información y orientación. Las pruebas son una de las mejores documentaciones que se pueden comprar. Las revisiones de código, si se hacen bien, pueden garantizar que tu equipo no dependa de formularios propensos a errores o de código demasiado ingenioso. En última instancia, el código debe escribirse para que lo lean los humanos.

A pesar de cómo se enseña a menudo, la programación es ante todo una actividad de comunicación, y no solo entre el programador y el compilador. No olvides que el ordenador entiende *cualquier* código (al menos si es sintácticamente correcto), pero eso no significa que un humano vaya a entender lo que tú intentas conseguir. Los mejores ingenieros de software se centran en escribir código que pueda ser entendido por los humanos que lo leen. Esto significa que tu código debe ser conciso y estar bien organizado. El código debe comunicar claramente su intención para que un humano pueda entender lo que se supone que debe hacer. Si es fácil de leer, será fácil de mantener, lo cual es crucial para el éxito a largo plazo de un proyecto de software.

Lo primero que aprendes cuando te inicias en el desarrollo de software es a programar. Probablemente con «¡Hola, mundo!». Lo que no siempre se enseña, ya sea en un curso universitario, un bootcamp o un

tutorial autodidacta, es qué hace que un código sea bueno, cómo escribir un buen código y cuándo escribir código para resolver un problema. Este capítulo pretende llenar ese vacío.

No reinventes la rueda

Antes de empezar a escribir línea tras línea de código (sin duda excelente), tómate un tiempo para ver si el problema en cuestión ya se ha resuelto. Los desarrolladores suelen escribir demasiado código, resolviendo problemas que otros ya han resuelto. Antes de empezar a escribir código nuevo, echa un vistazo a tu alrededor. ¿Hay alguna biblioteca que puedas aprovechar? Eso no es una licencia para añadir cosas ad hoc; no olvides analizar las dependencias de esas bibliotecas. Asegúrate de seguir las políticas y procedimientos de tu organización en relación con las bibliotecas de terceros.

Comprueba dos veces tu lenguaje de programación principal. ¿Hay alguna característica del lenguaje que puedas utilizar? Los lenguajes evolucionan (véase «La asignación de mayúsculas»); cinco minutos de búsqueda pueden ahorrarte horas de esfuerzo. ¿Qué hay de tus marcos de aplicación? Si crees que debería haber una forma mejor, es posible que la haya. Pregúntate: ¿eres la primera persona en el universo que ha escrito un código como este? Lo más probable es que no (véase «Adopta la filosofía del programador perezoso»).

Antes de invertir tiempo en escribir un nuevo fragmento de código, puede ser útil hacerse una serie de preguntas. Christopher M. Judd en-

seña el siguiente árbol de decisión en sus campamentos de entrenamiento (modifícalo para tu lenguaje y marcos de trabajo):

- ¿Se está haciendo esto en alguna parte del código base actual?
- ¿Ya lo hace el JDK?
- ¿Existe algún proyecto de Spring Framework o Spring Boot que resuelva este problema?
- ¿Existe alguna solución en Google Guava?
- ¿Existe alguna solución en Apache Commons?
- ¿Hay alguna otra biblioteca en el proyecto que resuelva este problema?
- ¿Existen bibliotecas de código abierto que resuelvan este problema?

Si has respondido «no» a todas las preguntas, puedes escribir el código, ¡asegurándote de utilizar pruebas a medida que avanzas! Si has respondido «sí» a alguna de las preguntas, enhorabuena, acabas de ahorrarte mucho tiempo; ¡aprovecha lo que hay disponible!

Dicho esto, las buenas prácticas de ayer suelen ser los antipatrones de mañana, y el hecho de que algo fuera lo correcto hace cinco o diez años no significa que siga siéndolo *hoy*. No tengas miedo de preguntar por qué, de cuestionar el statu quo. Es posible que te encuentres copiando sin pensar patrones o enfoques que encuentras o que te complazcas con el código generado por la IA sin comprender completamente *por qué*. El software avanza muy rápido; si no te detienes a mirar a tu alrededor de vez en cuando y adoptas deliberadamente una perspectiva abierta, podrías perderte cosas realmente importantes (véase «El valor de una nueva perspectiva»). Los lenguajes, los marcos,

las tecnologías y las técnicas están en constante evolución; mantenerse al día es algo habitual para un ingeniero de software. Haz preguntas incisivas y mantén la vista puesta en el horizonte.

¿Qué es un buen código?

Dejando todo eso a un lado, no siempre habrá una biblioteca o una característica del lenguaje que resuelva tu problema. Por supuesto, ¡escribirás código! Y puede que te preguntes: bueno, ¿qué es un buen código? Parafraseando a Potter Stewart, un buen código puede ser difícil de definir, pero lo reconoces cuando lo ves. Hay que reconocer que el concepto de código bueno frente a código malo puede ser muy subjetivo. Con el tiempo, empiezas a desarrollar un sentido para ello, e incluso puedes llegar a decir que el código tiene un olor. Por ejemplo, los métodos excesivamente largos suelen ser algo malo, pero aún así hay que examinar el código; puede que no haya un enfoque más sencillo. Pocos desarrolladores admitirán haber escrito código malo, pero cualquiera que haya abierto un editor es culpable de haber creado algún software que no es precisamente excelente.

Las métricas pueden proporcionar información sobre tu código base. Por ejemplo, la complejidad ciclomática puede indicarte el número de rutas dentro de tu código fuente, y un mayor número de rutas indica un código más complejo que probablemente se beneficiaría de una refactorización. Muchos lenguajes tienen analizadores de código fuent es como PMD, SonarQube y JSHint, que pueden ayudar a evitar que ciertos tipos de errores y malas prácticas de codificación infesten el código fuente. Herramientas como SonarQube y CodeScene pueden propor-

cionar información muy valiosa sobre tu código base. Es probable que tu organización tenga alguna. Si no estás seguro, pregunta a tus compañeros. Si no es así, ¿por qué no lideras la iniciativa de incorporar una a tu proyecto?

AÑADIR MÉTRICAS A PROYECTOS EXISTENTES

En un mundo perfecto, tu proyecto aprovecharía herramientas de análisis como las mencionadas desde el principio. Hacerlo permite que tu código «empiece limpio y se mantenga limpio» (suponiendo que tus compañeros solucionen los problemas detectados). Sin embargo, no siempre tendrás ese lujo y tendrás que añadir un linter o un analizador de código fuente a un proyecto que lleva meses o años en desarrollo.

Es tentador añadir una herramienta como PMD y activar todas las reglas. No lo hagas. En primer lugar, algunas de las reglas se contradicen entre sí; si una se cumple, otra fallará. En segundo lugar, activar demasiadas reglas suele dar lugar a un número inmanejable de advertencias, lo que es contraproducente. Puede desmoralizar al equipo. Con cientos o miles de advertencias, es difícil darse cuenta cuando alguien registra código que añade algunas más, y dedicar unas horas a corregir unas cuantas docenas de advertencias solo mueve el gráfico unos imperceptibles píxeles.

En su lugar, habla con tu equipo y elige unas cuantas reglas con las que todos estéis de acuerdo. Encontrar ese subconjunto puede resultar difícil. Activa esas reglas. Eso debería dar lugar a un número manejable de advertencias. Trabajad en equipo para solucionarlas. Una vez que lo hayáis hecho, activa un par de reglas más. Repite el proceso. Al cabo de unos meses, tendrás un amplio conjunto de reglas aplicadas a tu proyecto, así como un código más limpio.

Las métricas nunca deben seguirse sin pensar; ¡hay cierta complejidad que simplemente no se puede evitar! Debes aplicar el sentido común a cualquier regla general porque, en algunos casos, aplicar la regla común puede empeorar las cosas. Sin embargo, puedes aprovechar el efecto Hawthorne, que dice que las personas modifican su comportamiento cuando son observadas.[2] ¡Puedes utilizarlo en tu beneficio en un proyecto! Por ejemplo, si quieres que las pruebas cubran más código, muestra de forma destacada las estadísticas de cobertura del código.

AUMENTAR LA COBERTURA DEL CÓDIGO MEDIANTE MÉTRICAS

Aquí Nate de nuevo. Hace muchos años, me uní a un proyecto que llevaba varios meses produciendo código. Me sorprendió gratamente que, de hecho, hubiera pruebas, pero me decepcionó que todas, excepto unas pocas, fueran omitidas por la compilación porque «seguían fallando». Sin desanimarme, ejecuté un informe de cobertura de código que mostraba cuántas líneas de código se ejecutaban después de realizar todas las pruebas. Como era de esperar, era un número de un solo dígito. Pero lo publiqué y hablé de ello. Mucho. Durante la mayoría de las reuniones y retrospectivas. Cada vez que el número aumentaba, elogiaba a mis increíbles compañeros de equipo. Cuando alguien añadía pruebas a una parte especialmente complicada del código, les pedía que comentaran cómo lo habían hecho.

Poco a poco, nuestra cobertura de código fue aumentando y el código se volvió menos frágil. Tardamos varios meses, pero cuando pasé al siguiente proyecto, el código se acercaba al 70 % de cobertura. No estábamos satisfechos en absoluto, pero estábamos en una situación mucho mejor. Y todo empezó con la ejecución y la promoción de un simple informe de cobertura de código.

Las métricas pueden ser objeto de abuso o uso indebido (véase «Las métricas pueden ser engañosas»). Por ejemplo, muchas organizaciones han intentado evaluar al personal técnico por las líneas de código escritas, eliminadas o modificadas. Utiliza estas métricas con prudencia. Considera el acoplamiento externo o cuántas clases dependen de una clase determinada como otro ejemplo. En general, si observas un alto

grado de acoplamiento, lo considerarías una oportunidad para la refactorización. ¿Qué pasaría si esa clase fuera en realidad una fachada que esencialmente oculta un conjunto de clases? Debes contextualizar cualquier métrica.

El hecho de que puedas medir algo no significa que te proporcione información significativa. Intenta vincular las métricas a los objetivos del proyecto y céntrate en la tendencia de tus métricas. ¿Estás mejorando o empeorando con el tiempo? Da prioridad a los horizontes temporales cortos y estate preparado y dispuesto a ajustarte y adaptarte.

Añadir nuevas herramientas a tu proceso no es tu única opción. Muchos IDE proporcionan métricas de calidad del código, y hay un sinfín de complementos que puedes añadir a tu cadena de herramientas personal y que te ayudarán a escribir mejor código. Para acortar el ciclo de retroalimentación, muchas de las herramientas de análisis de código se pueden conectar a tu editor, notificándote las infracciones a medida que escribes, en lugar de esperar a que se produzca un error de compilación. Es posible que tu equipo tenga una configuración preferida. Si no estás seguro, pregunta a un compañero.

Menos es más

> *El objetivo del diseño de software es crear fragmentos o porciones que se adapten a la mente humana. El software sigue creciendo, pero la mente humana tiene un límite, por lo que tenemos que seguir fragmentando y dividiendo de forma diferente si queremos seguir haciendo cambios.*
>
> —Kent Beck

Dado el tamaño de algunas bases de código, podrías pensar que a los desarrolladores se les paga por carácter. Si bien algunos problemas realmente requieren millones de líneas para resolverse, en la mayoría de los casos es mejor optar por bases de código más pequeñas. Cuanto menos código, menos hay que cargar en el cerebro de las personas. Muchos proyectos alcanzan un tamaño tal que ya no es posible que un solo desarrollador comprenda *todo* el código, lo que es una de las fuerzas que ha dado lugar a microservicios y funciones como servicio.

Los típicos grandes líos suelen tener guías de inicio del tamaño de un diccionario y procesos de compilación que se miden en semanas y revoluciones de la luna. Los desarrolladores nuevos en el proyecto pueden tardar semanas o meses en ser productivos con el código. Cuanto más pequeño sea el código base, menos tiempo tardará un nuevo desarrollador en familiarizarse con el código y más rápido podrá empezar a contribuir; consulta el capítulo 6 para obtener más detalles. A continuación se indican algunos aspectos que debes tener en cuenta a la hora de controlar la longitud de tu código.

La ley cero de la informática

Muchas de las prácticas que adoptan los ingenieros de software en un esfuerzo por domar el código se reducen a la ley cero de la informática: alta cohesión, bajo acoplamiento. *La cohesión* es una medida de cómo se relacionan las cosas entre sí. Una alta cohesión significa, esencialmente, que las cosas similares están juntas. La función de notificación que también contiene lógica de impresión sería un ejemplo de baja cohesión. *El acoplamiento* se refiere al grado de interdependencia entre módulos o rutinas. El código con un acoplamiento estrecho puede ser difícil de modificar, ya que los cambios en una parte del código afectan inesperadamente a otras partes del sistema que parecen no estar relacionadas. Cambiar el servicio de notificación no debería romper el módulo de impresión.

Una alta cohesión y un bajo acoplamiento tienden a dar como resultado un código más legible y más fácil de mantener y evolucionar. Muchos patrones son formas de lograr una alta cohesión y un bajo acoplamiento, a menudo en diferentes niveles de abstracción. En el mejor de los casos, podría decirse que los microservicios son una alta cohesión y un bajo acoplamiento aplicados a los servicios.

Cuidado con el código repetitivo

El código repetitivo, incluso si es generado por un editor o un marco de trabajo, debe evitarse. Aunque no tengas que *escribirlo*, lo llevarás contigo durante toda la vida útil del proyecto. Las clases deben ser breves, de unas pocas páginas o menos.[3] Los lenguajes de programación

influirán en tu definición de *«breve»*, ya que algunos lenguajes son más prolijos que otros. En general, si tienes que desplazarte por la pantalla, es posible que tu clase sea demasiado larga.

Preferir la composición a la herencia

Muchos lenguajes permiten que las clases hereden de otras clases y, aunque esta característica puede ser muy potente y sin duda *existen relaciones* (un gato *es un* mamífero) en el software, tiende a utilizarse en exceso. Algunos desarrolladores utilizan la herencia como mecanismo de reutilización. La reutilización es un subproducto, no una justificación.

Veamos un ejemplo concreto. Supongamos que tu dominio tiene que ver con coches y camiones. Podrías crear una superclase de vehículos de la que desciendan tanto los coches como los camiones y que incluya un motor de combustión, ya que todos los coches y camiones tienen un motor de combustión, por lo que definirlo en la superclase garantiza que todos los coches y camiones también tengan un motor de combustión. Por ejemplo, veamos este pseudocódigo Ruby:

```
class Vehicle {
    Engine engine
    Integer num_wheels
    Integer num_doors
    Function brake {}
    Function accelerate {}
 }
```

```
class Truck < Vehicle {
    Number tow_capacity
}

class ElectricVehicle < Vehicle {
    Number range
    # wait… EVs don’t have engines…
}
```

Pero entonces aparecen los vehículos eléctricos, que no tienen motor de combustión. Un vehículo eléctrico es un vehículo, pero no como los *demás* vehículos. La composición es más flexible y debería preferirse a la herencia. Esto no quiere decir que *nunca* debas utilizar la herencia, sino que deberías preferir la composición.

Prefiere los métodos cortos

Escribe métodos cortos, de una sola línea de código. Al igual que una herramienta de línea de comandos de Linux o Unix, los métodos deben hacer una sola cosa, y deben hacerlo bien, trabajando en conjunto para lograr objetivos más amplios. Cualquier nombre de método que incluya conjunciones (y, o, pero) es una señal de que el método está haciendo demasiado. Los nombres de los métodos deben ser claros y concisos, y evitar ser ingeniosos. Si te cuesta nombrar un método, es posible que esté haciendo demasiado. Intenta dividirlo y ver qué pasa. Sé descriptivo. Elimina la duplicación de lógica, incluso en pequeñas cantidades. Simplifica y luego simplifica aún más.

En general, intenta crear bases de código más pequeñas y manejables para mejorar la comprensión y la productividad de los desarrolladores.

Escribe código para ser leído

Piensa en el desarrollador que seguirá tus pasos, el que tendrá que leer y mantener tu código. Recuerda aplicar la regla de oro a tu código (consulta «Aplicar la regla de oro al software»). ¿Cómo podrías escribir el código para facilitarle la vida al próximo desarrollador? ¿Qué necesitas comunicarle a la siguiente persona?

Hay una serie de pautas que puedes aplicar cuando se trata de la longitud «correcta» de una función, desde la reutilización hasta la elección de un número de líneas a menudo arbitrario. Martin Fowler ofrece un sabio consejo: ten en cuenta la separación entre la intención y la implementación. En otras palabras, ¿cuánto tiempo te lleva comprender lo que hace una función? Deberías poder leer el nombre y comprender inmediatamente lo que hace el código sin investigar el cuerpo del método en sí. Este principio a menudo dará lugar a funciones con solo unas pocas líneas de código, o incluso una sola; deja clara la *intención*. El nombre de la clase debería darte contexto, y los nombres de los métodos te ayudan a comprender lo que hace la clase.

El código se puede escribir como un artículo de periódico con una «pirámide invertida» . Los artículos comienzan con el lead, luego pasan a los hechos clave y luego a un contexto más profundo. Tú, como lector, puedes detenerte en el nivel de detalle que desees, tal vez solo en los

primeros párrafos, tal vez hasta el final. Como se dice en el mundo editorial, no entierres el lead.

El código debe seguir el mismo modelo. El nombre de la clase es casi como el titular de un artículo. A partir de ahí, deberías poder echar un vistazo a las firmas de los métodos para obtener una comprensión general. Si quieres explorar una función o una llamada a otra clase, eres libre de hacerlo, pero debes seguir entendiendo la esencia del código.

NOMBRAR COSAS ES DIFÍCIL

Todos los desarrolladores se han quedado alguna vez mirando fijamente su editor, luchando por encontrar un nombre para una variable, un método o una clase. Y aunque esta dificultad puede indicar una comprensión insuficiente del problema o un código demasiado complejo, hay una razón por la que `foo`, `bar` y `foobar` son casos tan comunes. Encontrar nombres significativos puede llevar mucho tiempo. ¡No te apresures! Una vez más, tómate el tiempo necesario para que te lleve menos tiempo. Vale la pena el esfuerzo de pensar en buenos nombres. No tengas miedo de pedir la opinión de un compañero de equipo. A veces, con solo explicar en qué estás trabajando será suficiente para inspirar el nombre perfecto.

¡Tu IDE puede ayudarte! Muchos desarrolladores trabajan hacia atrás a partir de la declaración, permitiendo que el editor haga sugerencias. En otras palabras, empieza con un nuevo `Arrays.asList({"Red", "Green", "Blue"});`. Tu IDE puede utilizar un completado posfijo para sugerir un nombre de variable. Las herramientas de codificación agenticas también pueden proporcionar algo de inspiración.

No dudes en refactorizar el código mal nombrado. Los editores modernos cuentan con potentes herramientas que facilitan el cambio de nombre. Solo asegúrate de no cambiar constantemente los nombres de los conceptos básicos del dominio, ya que eso suele indicar un problema de concepción errónea.

También puede ser útil jugar al juego de las palabras sin sentido. A menudo, la primera palabra que utilizas para definir un concepto no es la mejor opción, pero puede moldear tu forma de pensar sobre el domi-

nio del problema. Cuando trabajes por primera vez en el dominio, ¡inventa una palabra! Después de realizar algunos análisis adicionales, sustituye las palabras sin sentido por palabras reales. Probablemente se te ocurra algo muy diferente y más claro que tu primera reacción. La próxima vez que te quedes realmente atascado sobre cómo llamar a algo, utiliza algunas palabras sin sentido y vuelve atrás.

El problema de los comentarios en el código

Todos los lenguajes de programación tienen *alguna* facilidad para que los desarrolladores se comuniquen, no con el compilador o el tiempo de ejecución, sino con sus compañeros desarrolladores a través de un comentario de código. Y aunque pueda parecer que se debe comentar libremente el código, podría decirse que hacerlo es un código sospechoso.[4] Los comentarios en el código violan el principio DRY, , también conocido como «Don't Repeat Yourself» (no te repitas). Aunque lo más habitual es ver cómo se viola el principio DRY con código copiado y pegado o plagado de duplicaciones lógicas, los comentarios también pueden ser problemáticos. En la mayoría de los casos, escribes el código y luego te das la vuelta y escribes sobre el código.

Los comentarios en el código también pueden ser un dolor de cabeza a la hora del mantenimiento. Modificas el código, pero ¿te tomarás el tiempo de actualizar también los comentarios? A menudo, los comentarios comienzan a divergir del código, lo que añade otra capa de sedi-

mento a los desarrolladores que se encuentran con el código meses o años más tarde. El código debe escribirse de forma que sea legible. Es mejor dedicar tu tiempo a hacer que el código sea más fácil de leer que a documentar lo que hiciste. Los lenguajes expresivos sin duda ayudan a conseguir un código legible, y es posible que quieras evitar las características «mágicas» más matizadas del lenguaje que elijas. Los buenos nombres de métodos y variables son mejores que los comentarios; si crees que necesitas escribir un comentario, intenta cambiar el nombre del método o la variable. Si lo haces, casi siempre se evitará la necesidad de un comentario.

Podría decirse que los comentarios menos útiles son los bloques de cambios en el código, que suelen ser las primeras cientos (o miles) de líneas de un archivo. Y aunque pueden ser un punto de referencia interesante en tu viaje arqueológico, la repetición de números de seguimiento, fechas, nombres y párrafos sobre el cambio añade ruido al proceso. Deja que tu herramienta de gestión de código fuente haga su trabajo.

Eso no quiere decir que el código *nunca* deba tener comentarios.[5] Si estás haciendo algo que no es obvio y no has encontrado una forma de simplificar el código, los comentarios que expliquen la situación pueden ser útiles. Estos comentarios deben centrarse en *por qué* hiciste algo en lugar de *en lo que* hiciste. También puede ser útil proporcionar contexto sobre la elección de un algoritmo en lugar de otro o explicar una parte de la lógica particularmente compleja. Dicho esto, si el código necesita ser explicado, reescríbelo.

Los comentarios pueden servir como recordatorios para nuestro yo futuro, o como advertencia de que un truco funciona pero (todavía) no entiendes por qué. Algunos desarrolladores dejan comentarios como advertencias para futuros desarrolladores, como en el siguiente ejemplo:

```
// Dear maintainer:
//
// Once you are done trying to
// “optimize” this routine,
// and have realized what a terrible
// mistake that was,
// please increment the following
// counter as a warning
// to the next person:
//
// total_hours_wasted_here = 42
```

¿Son buenos estos comentarios? Depende, y es probable que muchos de los ejemplos que oigas sean apócrifos. Dicho esto, no dudes en dejar consejos para tu yo futuro.

Las pruebas como documentación

Si los comentarios no son una forma adecuada de documentar el código, ¿qué debes hacer? Escribir pruebas. Las pruebas, especialmente las escritas en estilos más fluidos, son especificaciones ejecutables que evolucionan junto con el código de producción.[6] La documentación, ya

sean comentarios de código, archivos README o especificaciones, tiende a divergir del código tan pronto como se escribe. Las pruebas escritas mientras escribes código te permiten refactorizar libremente y aumentar tu confianza en la calidad de tu aplicación. También actúan como señales para los desarrolladores que te siguen. Las pruebas se exploran con más profundidad en el capítulo 5.

Añadir pruebas al código existente te permite capturar lo que estás aprendiendo sobre cómo funciona el código y desbloquear tus conocimientos para que otros desarrolladores puedan beneficiarse de tu trabajo. Al renombrar un método o una variable y podar algo de código muerto, estás dejando el código mejor de lo que lo encontraste.

Algunos desarrolladores insisten en que necesitan escribir abundantes comentarios para quienes consumen sus servicios. Si bien estos comentarios son menos molestos que los mencionados anteriormente, no son tu única opción. Una vez más, las pruebas constituyen un mecanismo de documentación más resistente. El uso de contratos impulsados por el consumidor te permite transmitir lo que hace tu servicio y, al mismo tiempo, te da la confianza necesaria para iterar según sea necesario. Siempre que no hayas incumplido el contrato, puedes desarrollar tu código sin preocuparte por romper inadvertidamente un sistema posterior. Los consumidores ganan confianza en tus servicios, ya que disponen de un conjunto de pruebas que pueden ejecutar y que simulan el comportamiento esperado de tu código. Pueden modificar su código sin temor a introducir un nuevo defecto.

Los contratos impulsados por el consumidor son una parte vital de un software fiable y resistente. Muchos lenguajes y marcos tienen proyec-

tos que puedes (¡y debes!) aprovechar con tus aplicaciones. Desde Spring Cloud Contract hasta versiones de Pact para casi todas las plataformas, tienes opciones.

Evita el código ingenioso

El software es difícil y los ámbitos en los que trabajas son complejos. Pero no toda la complejidad es igual. En su ampliamente citado ensayo No Silver Bullet, y *esencial*, Fred Brooks establece una distinción entre complejidad accidental y esencial. En resumen, *la complejidad esencial* es inherente al software, desde los matices de las reglas de negocio hasta la comunicación con tu equipo, pasando por la naturaleza siempre cambiante de una base de código. No hay nada que puedas hacer para eliminar esta complejidad del software; es parte del trabajo. Por otro lado, *la complejidad accidental* son las formas en que los desarrolladores complican las cosas más de lo necesario, desde tecnologías ruidosas hasta herramientas pesadas. En caso de duda, mantén la simplicidad.

El software no es inmune a los proverbiales productos milagrosos. Muchas empresas intentan vender productos o procesos que revolucionarán la entrega de software. Vale la pena ser escéptico. Por supuesto, hay oportunidades de mejora, pero el método científico nos recuerda que las afirmaciones extraordinarias requieren pruebas extraordinarias. Debes estar atento para eliminar la complejidad accidental siempre que sea posible. Consulta el capítulo 6 para obtener una explicación más detallada.

Los lenguajes y marcos de trabajo suelen tener formas e es propensas a errores. Por ejemplo, echa un vistazo al siguiente código Java. ¿Puedes detectar el problema?[7] `if` En Java, los corchetes son *técnicamente* opcionales en un bloque de instrucciones único. Ahora bien, algunos desarrolladores podrían argumentar a favor de omitir los corchetes, ya que es menos prolijo y las líneas (esencialmente) vacías son un desperdicio de espacio vertical. Pero, ¿qué sucede cuando otro desarrollador añade una segunda instrucción? ¿Recordarán añadir los corchetes o se dejarán seducir por la indentación del código?

```
if (condition)
  doFoo();
  doBar();
```

Evita las formas propensas a errores en la cadena de herramientas que elijas. El hecho de que tú lo entiendas claramente no garantiza que la información se distribuya ampliamente entre tu equipo. No tengas miedo de actualizar (o establecer) estándares de codificación para cubrir estos casos. Existen varias herramientas de análisis estático que puedes añadir a tu canal de implementación para evitar que tú y tu equipo introduzcáis inadvertidamente este tipo de problemas. Aprovecha sus ventajas.

Revisiones de código

Las revisiones de código pueden variar desde pedir a un compañero su opinión sobre un método hasta sesiones de varias horas con varios

desarrolladores. Independientemente de los detalles específicos de la implementación, las revisiones de código son una forma excelente de aprender, compartir experiencias y socializar conocimientos. Cuantas más personas revisen el código, mejor, y esa es una de las razones por las que algunas organizaciones utilizan la programación en pareja.

Ya sean formales o no, ciertas prácticas pueden mejorar tu revisión de código. En primer lugar, no seas sarcástico. Evita el sarcasmo. Pedir opiniones puede ser muy estresante para algunas personas, y muchas se toman las críticas como algo personal. La forma en que compartes tus comentarios es fundamental. Sé empático con tus compañeros de equipo. Aunque puede ser tentador utilizar la revisión de código como una oportunidad para soltar alguna curiosidad esotérica a tu equipo, el objetivo es mejorar el código, no exhibir tus conocimientos técnicos.

> *La única forma de crear algo grande es reconocer que quizá aún no sea grande. Tu objetivo es encontrar la mejor solución, no medir tu autoestima personal en función de ella.*
>
> —Jonas Downey, diseñador de software, desarrollador y escritor

Centra tu atención en lo más importante. Aunque los aspectos estilísticos son importantes, es mejor que dediques tus esfuerzos a los niveles inferiores de la pirámide de revisión de código. Puedes (y debes) automatizar las cuestiones relacionadas con el formato y el estilo;[8] deja que un ordenador se encargue de eso. Tu tiempo y esfuerzo deben dedicarse a las cosas que los ordenadores no pueden detectar por ti. ¿Son claros y concisos los nombres de los métodos y las variables? ¿Es legi-

ble el código? ¿Hay duplicaciones? ¿Tiene el código el registro, el seguimiento y las métricas adecuados? ¿Son coherentes las interfaces con el resto del código? ¿Ha utilizado el desarrollador algún formulario propenso a errores? ¿Es correcto el modelo? ¿Han elegido una abstracció errónea?

Evita la revisión de código por casillas

En algunos casos, las revisiones de código son poco más que una casilla de verificación en el sistema de gestión del código fuente. Algunas organizaciones exigen que todo el código se revise antes de fusionarlo con la línea principal. Ese objetivo puede ser loable, pero en la mayoría de los casos se traduce en que un desarrollador le pida a otro que «revise» el código, lo que normalmente significa marcar la casilla por su compañero. Aunque parta de una buena intención, este tipo de revisiones van en contra del objetivo. Trabajar en lotes pequeños hace que las revisiones sean más sencillas y eficaces.

Algunas organizaciones utilizan solicitudes de incorporación de cambios (PR) como forma de garantizar la calidad del código. Aunque pueden resultar más cómodas que pasar la tarde en una videollamada discutiendo sobre el código, las PR no siempre favorecen la cohesión y la confianza del equipo. Algunos desarrolladores utilizan las PR como una oportunidad para buscarle tres pies al gato, iniciar una guerra territorial o reavivar un tema que ya se había cerrado. Las PR también son vulnerables a la respuesta «LFTM» (looks fine to me, «a mí me pa-

rece bien») del , que puede ser aceptable en algunos casos, pero que cae víctima de la revisión de casillas mencionada anteriormente.

Los comentarios basados en texto también carecen del tono y el lenguaje corporal de las conversaciones cara a cara. Puede que no *sea tu intención que* un comentario se interprete de forma ácida o mordaz, pero la persona al otro lado de la solicitud puede interpretarlo así. No te sorprendas si algunos de tus compañeros de equipo, especialmente los menos experimentados, temen las PR. Una vez más, practica la empatía. Pregúntate cómo te gustaría que te trataran y actúa en consecuencia. El personal superior debe dar ejemplo.

Es difícil ser criticado

Los desarrolladores suelen invertir mucho de sí mismos en su trabajo, por lo que a veces es difícil no tomarse los comentarios como algo personal. Las revisiones de código no son una oportunidad para avergonzar a alguien porque no conozca alguna nueva característica del lenguaje o no haya visto inmediatamente una forma más sencilla de resolver un problema. Nadie es perfecto; todo el mundo comete errores. Las revisiones de código tienen como objetivo crear mejores aplicaciones y se centran en el código, no en el programador. No te tomes las revisiones de código como algo personal. Sé humilde y haz preguntas útiles. Las críticas son más fáciles de digerir cuando van acompañadas de elogios, así que asegúrate de señalar también los aspectos *positivos*.

Comparte tus experiencias. Las historias personales tienen un gran peso y difuminan la resistencia natural de las personas al cambio.

Ofrece ayuda con cosas que hayas encontrado en proyectos anteriores. Ten cuidado con las afirmaciones generales. Asegúrate de tener todos los detalles antes de declarar que algo no funcionará, ya que es posible que te estés perdiendo un dato clave del contexto. ¿Hay algún antecedente que no conozcas? Quizás haya restricciones que desconoces. Cíñete al código.

> *Cada uno de nosotros está haciendo lo mejor que puede dado nuestro estado de conciencia.*
>
> —Deepak Chopra, autor indio-estadounidense y defensor de la medicina alternativa

Si algo en el código de alguien te preocupa, no tengas miedo de hablar directamente con el desarrollador. Las personas pueden ponerse muy a la defensiva, especialmente en situaciones grupales. Una breve conversación cara a cara podría ser la solución. No embosques a un compañero de equipo; nadie sale ganando en esas interacciones. Recuerda que las revisiones son una oportunidad para aprender y para enseñar.

Fomentar la confianza

Si no debes hacer revisiones con casillas de verificación o LFTM, ¿qué debes hacer? Independientemente de tu proceso de revisión de código, no pierdas de vista su propósito. Debes compartir experiencias, aprender y crecer como equipo, evitando prácticas problemáticas como expresiones confusas o enfoques obsoletos. Anima a tus compañeros de equipo a hacer preguntas y a ofrecer comentarios constructivos. Las

revisiones de código deben fomentar la propiedad colectiva del código y la confianza entre el equipo. Promocionar el error de la semana o dedicar tiempo a compartir algo con lo que te has encontrado puede ser increíblemente eficaz.

Una reunión periódica en la que se anime a las personas a hablar sobre un defecto interesante que hayan resuelto es una técnica de aprendizaje muy valiosa. Es posible que asumas que todo el mundo sabe qué hacer cuando se encuentra con un problema concreto, pero discutir esas situaciones con tus compañeros de equipo permite difundir el conocimiento. El simple hecho de dedicar un tiempo durante la reunión del viernes a discutir algo interesante que haya ocurrido durante la semana puede dar sus frutos.

No hace falta decirlo, pero trata a tus compañeros de equipo con respeto. Sé amable, haz lo correcto, haz lo que funcione. No tengas miedo de tomarte un momento para revisar tu enfoque y preguntarte si podría haber una forma mejor de hacerlo. Tanto si sigues una metodología de desarrollo ágil como si no, debes adaptarte y ajustarte regularmente. Si algo no funciona, ¡ , cámbialo!

Aprender nuevos lenguajes

Si quieres mejorar en la escritura de código, necesitas, bueno, escribir código. Pero puedes acelerar esas habilidades aprendiendo nuevos lenguajes de programación. Piensa en aprender un idioma extranjero. Durante meses, incluso años, traducirás ese idioma a tu lengua materna. Con el tiempo, pensarás, incluso soñarás, en tu nuevo idioma. Pero lle-

va tiempo. Los lenguajes de programación no son diferentes. Bueno, tienen reglas gramaticales muy exigentes y muchas menos palabras clave.

La clave es la inmersión. Si realmente quieres aprender francés, mudarte a París acelerará tu progreso. Aprender un nuevo lenguaje de programación es muy similar. Crea una aplicación en el nuevo lenguaje que resuelva un problema que te haya afectado en el trabajo o en casa. Sigue a la comunidad en las redes sociales. Escucha los podcasts relacionados o mira los videos de la última conferencia centrada en el lenguaje. Mejor aún, asiste al grupo de usuarios local o ve al próximo evento sobre el tema.

Los desarrolladores tienden a encariñarse mucho con su primer lenguaje. Ten cuidado con esto. Los lenguajes de programación son solo herramientas. Al igual que un martillo es una herramienta mejor para clavar un clavo que un destornillador, algunos lenguajes se adaptan mejor que otros a determinados problemas. Por ejemplo, muchos sistemas integrados están escritos en C, ya que es muy portátil y fiable y se puede optimizar en gran medida para plataformas específicas. Acostúmbrate a mirar, explorar y aprender otros lenguajes. Algunas personas influyentes sugieren aprender un nuevo lenguaje cada año o cada dos años.

LA PROGRAMACIÓN SE BASA FUNDAMENTALMENTE EN LA COMUNICACIÓN

Antes de que los departamentos de informática de la generac e comenzaran a surgir en las universidades de todo el mundo, la programación solía pertenecer al departamento de matemáticas, y muchos asumían que la aptitud matemática era un requisito previo para tener éxito en el software. En algunos casos, las universidades todavía utilizan exámenes de matemáticas para seleccionar a los estudiantes. A pesar de esta suposición, las investigaciones demuestran que la aptitud *lingüística* es un indicador mucho mejor de la rapidez con la que alguien aprende un nuevo lenguaje de programación que la habilidad en matemáticas. Si bien ciertos ámbitos pueden requerir muchos conocimientos matemáticos, el arte de la programación no es uno de ellos.

La programación es, ante todo, una actividad de comunicación, y no entre el programador y el compilador. No olvides que el ordenador entiende *cualquier* código (al menos si es sintácticamente correcto), pero eso no significa que un humano vaya a entender lo que tú intentas lograr. Los mejores ingenieros de software se centran en la persona que lee el código. Los buenos escritores (de cualquier tipo) siempre tienen en mente a su público.

Aprender un nuevo lenguaje lleva tiempo. ¿Cómo justificas la inversión que supone aprender algo que quizá no utilices a diario en el trabajo? Aprender un nuevo lenguaje cambiará tu forma de programar, incluso si no llegas a utilizar la nueva herramienta día tras día. Cuando busques un nuevo reto lingüístico, intenta elegir algo que sea diferente de lo que utilizas en el trabajo. Si eres un desarrollador Java con

experiencia, busca más allá de otros idiomas similares al C, como C# (no es que haya nada malo en aprender C#, claro está), y dirígete hacia paradigmas diferentes. Considera en su lugar un lenguaje dinámico como Ruby o un lenguaje funcional como Haskell. Confía en nosotros, incluso un examen superficial de un lenguaje fuera de tu ámbito habitual alterará fundamentalmente tu enfoque de la programación. Es posible que llegues a apreciar más tu lenguaje habitual o que te encuentres escribiendo código de una manera diferente. Tómate tu tiempo para aprender cosas nuevas.

Aprender nuevos lenguajes se vuelve más fácil con el tiempo. Cuantos más lenguajes conozcas, más podrás comparar. Piensa en el primer lenguaje que aprendiste: empezaste desde cero. Para cuando llegas al tercer, cuarto o quinto lenguaje, empiezas a ver cómo una expresión es igual que otra de Java, o cómo alguna estructura se ha tomado prestada de Ruby.

Al principio de tu carrera, debes centrar tu atención en profundizar todo lo posible en los lenguajes y marcos que utilizas a diario. Sin embargo, no descuides la exploración de otras opciones. Hacerlo no solo te revitalizará, sino que también te convertirá en un mejor desarrollador.

Conclusión

Los desarrolladores escriben código; es parte integrante de su trabajo. Evitar las formas propensas a errores y el código excesivamente ingenioso puede marcar la diferencia entre un código base con el que es un

placer trabajar y uno que los desarrolladores evitan como la peste. Desde revisiones de código hasta herramientas de análisis, hay muchas formas de ayudarte a escribir mejor código. Prefiere escribir pruebas en lugar de comentarios abundantes. Cuando critiques el código, sé empático. Nunca olvides que el código debe escribirse para que lo lean los humanos; adherirse a ese principio contribuye en gran medida a garantizar que escribas código que otros calificarán con la elusiva etiqueta de «bueno».

Escribir buen código es un proceso de aprendizaje continuo. Mantente abierto a aprender nuevas habilidades y mejorar las que ya has desarrollado. Mantenerte al día sobre las buenas prácticas en ingeniería de software es clave para tu éxito a largo plazo en el campo.

Ponerlo en práctica

No hay atajos; si quieres mejorar como desarrollador, ¡tendrás que escribir código! Como dice el viejo chiste: un peatón en la calle 57 ve a un músico bajarse de un taxi y le pregunta: «¿Cómo se llega al Carnegie Hall?». Sin pausa, el artista responde con cansancio: «Practicando». Si quieres ser un mejor desarrollador, necesitas practicar.

Considera la posibilidad de añadir un poco de estudio a tu rutina. Periódicamente, digamos cada dos meses más o menos, reserva un par de horas para abordar un código kata de . En las artes marciales, los katas son una serie de bloqueos, patadas y puñetazos que los alumnos estudian y repiten innumerables veces. Los códigos katas trasladan esta idea al software, proporcionando problemas sencillos que te dan

la oportunidad de practicar tu oficio y trabajar en cosas con las que quizá no te encuentres en tu día a día como programador.

Puedes aprovechar los katas de código de muchas maneras. Puedes elegir uno y resolverlo en dos o tres lenguajes de programación. Puedes formar equipo con un amigo o colega para resolver un kata. Hay múltiples formas de resolver un kata determinado; ponte a prueba para resolver el mismo kata de dos o tres maneras diferentes. Realiza una revisión del código de un kata que hayas resuelto hace unos meses (¡o años!): ¿qué harías de manera diferente hoy?

Una vez más, puedes aprovechar el universo del software de código abierto. Reserva un par de horas para contribuir a una biblioteca de código abierto que utilices (o quieras utilizar). Si no estás seguro de por dónde empezar, echa un vistazo a los repositorios de tendencia en GitHub. Contribuir al código abierto es un excelente laboratorio de aprendizaje, y no es tan difícil empezar como podrías pensar.[9]

Aún así, puedes aprender mucho sobre cómo escribir buen código sin contribuir. Elige un proyecto y dedica un par de horas a leer el código. ¿Qué te gusta? ¿Qué no te gusta? ¿Qué harías de manera diferente? Ejecuta un analizador de código fuente sobre el código: ¿qué te dice sobre el proyecto? Si ves algún problema evidente, ¡no tengas miedo de plantearlo o contribuir con un PR!

Mantenerse al día con los cambios es un componente fundamental para una carrera exitosa en el ámbito del software; haz que sea un hábito actualizar tus conocimientos sobre los lenguajes y marcos de trabajo básicos. Es posible que no encuentres nuevas funciones de forma

orgánica en tu trabajo diario, así que dedica unas horas una o dos veces al año a ver qué se ha añadido a tu kit de herramientas. La mayoría de las tecnologías tienen defensores o promotores, así que síguelos o suscríbete para estar al tanto de los cambios. No todas las novedades funcionarán para tus aplicaciones, al menos hoy en día, pero es más sencillo asimilar unas pocas actualizaciones periódicamente que intentar aprender docenas o cientos de cosas nuevas cada pocos años.[10]

Por último, pero no por ello menos importante, considera la posibilidad de ofrecer tus habilidades de programación como voluntario a una organización benéfica o sin ánimo de lucro local. Muchas organizaciones que lo merecen buscan constantemente la ayuda de la comunidad técnica. El voluntariado puede darte la oportunidad de practicar tu oficio, tal vez utilizando un lenguaje o un marco que estés tratando de aprender, al tiempo que ayudas a una causa que te interesa.

Recursos adicionales

- «Retrato de un novato», de Steve Yegge
- Hilo de debate sobre la preferencia de la composición frente a la herencia
- *El mítico mes-hombre: Ensayos sobre ingeniería de software*, edición aniversario, de Frederick P. Brooks (O'Reilly, 1995)
- «Un uso adecuado de las métricas», por Patrick Kua
- «Simple Made Easy», de Rich Hickey

[1] Actualmente, la IA no distingue entre si puede escribir código para resolver un problema y si debe hacerlo, y es probable que nunca lo haga.

[2] Como en la autopista, cuando la gente ve al policía estatal en la mediana y todo el mundo reduce la velocidad.

[3] Aunque con los tamaños de monitor modernos, es posible que quieras limitarte a una sola página.

[4] A menudo se puede saber el nivel de experiencia de un desarrollador por su uso (o evitación) de los comentarios en el código.

[5] El dogmatismo, ya sea en el software o en la vida, rara vez es el camino correcto. Es mejor optar por el pragmatismo.

[6] Bibliotecas como Spring REST Docs también pueden ayudar a que la documentación se mantenga sincronizada con el código.

[7] Uno de tus autores, cuyo nombre no revelaré en este caso, pasó la mayor parte de dos semanas depurando ese bloque de código.

[8] También conocidas como revisiones «me parece bien», no son un buen uso del tiempo de una persona.

[9] Muchos proyectos tienen listas de errores marcados como «para colaboradores noveles», pero no tengas miedo de ponerte en contacto con los colaboradores; la mayoría estará encantada de ayudarte a ponerte en marcha.

[10] Algo que también se aplica a *las actualizaciones* de dichas tecnologías...